

Taller 5: Empaquetamiento

Instrucciones:

En este taller emplearemos algunas herramientas para el empaquetamiento de modelos de analítica en python.

La entrega de este taller consiste en un reporte en formato de documento de texto, donde pueda incorporar fácilmente capturas de pantalla, textos y elementos similares. Puede utilizar formatos como Word, LibreOffice, Markdown, u otros. El reporte debe entregarse en formato PDF.

Parte 1: desarrollo de modelos

1. Junto a este enunciado encontrará el cuaderno *bank-churners-train-test.ipynb* y los datos *Bank-Churn.csv*.
2. Abra estos archivos localmente, ejecute el cuaderno y explore en detalle su ejecución.
3. Este cuaderno puede tomarse como el resultado final de un proceso de exploración que nos ha llevado al modelo que allí se entrena y prueba. Asegúrese de comprender los pasos que se realizan en el cuaderno en detalle.

Parte 2: preparación del empaquetamiento

1. Junto a este enunciado encontrará el archivo *package-src.zip*.
2. Descomprima este archivo localmente. Encontrará una estructura de carpetas con múltiples archivos.
3. Abra el archivo *setup.py* y familiarícese con su estructura y contenido.
4. Abra el archivo *tox.ini* y familiarícese con su estructura y contenido. En este archivo se definen dos ambientes clave que usaremos hacia adelante: *test_package* y *train*. En cada uno de estos ambientes se define un directorio para el ambiente, un conjunto de dependencias y los comandos a ejecutar.
5. En la carpeta *model* encontrará el archivo *train_pipeline.py*, que ejecuta el entrenamiento. Allí se emplea el pipeline de datos definido en el archivo *pipeline.py* y se entrena el modelo y guarda el modelo.
6. Note que en éste y otros scripts se hace referencia al archivo *config.yml*, el cual define el valor que toman algunos parámetros clave. Explore este archivo con detenimiento y asegúrese de entender los parámetros definidos.

7. En la carpeta `model` encontrará el archivo `predict.py`, que realiza predicciones. Note que para esto se emplea también el pipeline de datos definido en el archivo `pipeline.py` y la validación de datos disponible en el archivo `processing/validation.py`.
8. Revise el archivo `processing/validation.py`, donde encontrará los esquemas de un dato `DataInputSchema` y de múltiples datos `MultipleDataInputs`.
9. Explore la carpeta `requirements`, que contiene las dependencias del paquete (`requirements.txt`), las de prueba (`test_requirements.txt`) y las tipos (`typing_requirements.txt`).
10. En la carpeta `tests` encontrará los archivos de prueba. En `conftest.py` la configuración inicial que define el archivo de prueba. Y en `test_prediction.py` una prueba unitaria del método de predicción. Asegúrese de comprender qué se hace en el archivo `test_prediction.py` en detalle.
11. Note que en este paquete contamos con esencialmente los mismos pasos que en el cuaderno de jupyter, pero estructurados de una manera adecuada para empaquetamiento.

Parte 3: empaquetamiento

1. Lance una instancia en AWS EC2. Se recomienda una máquina **t3.small**, con sistema operativo **Ubuntu 24.04** y **20GB** de disco. **Incluya un pantallazo de la consola de AWS EC2 con la máquina en ejecución en su reporte. Su usuario de AWS debe estar visible en el pantallazo.**
2. Cree un repositorio en Github y clónelo localmente. Descomprima allí el archivo `package-src.zip`. En su repositorio debe quedar una carpeta `package-src` con los contenidos de este archivo (carpetas `model`, `requirements`, `test`, entre otros). Cree también (al lado de la carpeta `package-src`) una carpeta `test`, que contenga los archivos `test-package.py` y `bankchurn_test.csv`, que encontrará adjuntos a este taller. La estructura de carpetas de su repositorio debe quedar así:
 - `package-src`
 - `model`
 - `requirements`
 - `tests`
 - `test`

Realice un `commit+push` para actualizar el repositorio.

3. Conéctese a la máquina:
 - a) Abra una terminal: En windows, escriba `cmd` y `Enter`. En macOS, abra la aplicación llamada *Terminal*.
 - b) En la terminal emita el comando

```
ssh -i /path/to/llave.pem ubuntu@IP
```

donde `/path/to/` se refiere a la ubicación del archivo `llave.pem` que descargó, e `IP` es la dirección IP de la instancia EC2 que lanzó. Si prefiere, en la terminal puede navegar a la ubicación del archivo `llave.pem` y emitir el comando

```
ssh -i llave.pem ubuntu@IP
```

Note que usamos en este caso ubuntu, pues éste es el usuario creado por defecto como administrador con sistema operativo Ubuntu server. Incluya en su reporte un *screenshot* de la conexión a la máquina virtual.

4. Actualice las ubicaciones de los paquetes con el siguiente comando:

```
sudo apt update
```

5. Instale pip para python3

```
sudo apt install python3-pip
```

6. Instale unzip para manipular los archivos .zip

```
sudo apt install zip unzip
```

7. Instale venv para crear ambientes virtuales con el siguiente comando:

```
sudo apt install python3.12-venv
```

8. Cree un ambiente virtual con el nombre env-tox

```
python3 -m venv /home/ubuntu/env-tox
```

9. Active el ambiente con el comando:

```
source env-tox/bin/activate
```

10. Clone el repositorio en la máquina virtual. Verifique que dentro de la carpeta principal del repositorio haya quedado la carpeta package-src y la carpeta test.

11. Instale tox, una librería de automatización y pruebas

```
pip install tox
sudo apt-get install tox
```

Incluya la dirección /home/ubuntu/.local/bin en el PATH para facilitar su ejecución

```
PATH=$PATH:/home/ubuntu/.local/bin
```

12. Desde la carpeta del repositorio, ingrese a la carpeta package-src

```
cd package-src
```

y ejecute el ambiente de entrenamiento de tox

```
tox run -e train
```

Incluya en su reporte un *screenshot* de la salida de este comando. Asegúrese de comprender qué realiza el comando y verifique que se pasen todas las pruebas.

13. Ejecute ahora el ambiente de prueba del paquete de tox

```
tox run -e test_package
```

Incluya en su reporte un *screenshot* de la salida de este comando. Asegúrese de comprender qué realiza el comando y verifique que se pasen todas las pruebas.

14. Con todas las pruebas pasadas, procederemos a construir el paquete, para esto necesitamos instalar el paquete build

```
python3 -m pip install --upgrade build
```

y construimos el paquete con el comando

```
python3 -m build
```

Note que se crea una carpeta **dist** en donde se tienen archivos para la instalación del paquete (uno tar.gz y uno whl). **Incluya en su reporte un *screenshot* donde se evidencien los archivos creados en esta carpeta.**

15. Ingrese a la carpeta test y copie allí el archivo whl del paquete creado

```
cd ..  
cd test  
cp ../package-src/dist/model_abandono-0.0.1-py3-none-any.whl .
```

Note el punto al final del comando.

16. Instale el paquete

```
pip install model_abandono-0.0.1-py3-none-any.whl
```

Note que se instalan todas las dependencias del paquete.

17. Ejecute el siguiente comando para mostrar los paquetes instalados:

```
pip freeze
```

Incluya en su reporte un *screenshot* donde se evidencien los paquetes con el paquete del modelo instalado.

18. Ejecute el archivo test-package.py

```
python3 test-package.py
```

Incluya en su reporte un *screenshot* donde se evidencie la ejecución de este comando.

19. Realice un commit+push para actualizar el repositorio.
20. En su máquina local, actualice el repositorio.

Parte 4: modificando el paquete

1. Localmente, modifique su modelo para usar todas las características excepto una de su elección. Para ello, edite el archivo *config.yml* eliminando uno de los elementos en features y agregándolo a temp_features. **Incluya en su reporte un pantallazo de la modificación realizada al archivo config.yml.**
2. Modifique el archivo VERSION de tal forma que ahora corresponda a la versión 0.0.2.
3. Realice un commit+push para actualizar el repositorio.
4. En la máquina virtual, actualice el repositorio. **Incluya en su reporte un pantallazo del archivo VERSION actualizado en la máquina virtual.**

- Con esta versión actualizada, realice nuevamente desde el paso 12 hasta el 20 de la parte 3 para verificar que se pasan adecuadamente los pasos de prueba y empaquetamiento. Desde la carpeta del repositorio en la máquina, los pasos serían:

```
cd package-src
tox run -e train
tox run -e test_package
python3 -m build
cd ..
cd test
cp ../package-src2/dist/model_abandono-0.0.2-py3-none-any.whl .
pip install model_abandono-0.0.2-py3-none-any.whl
pip freeze
python3 test-package.py
```

Incluya en su reporte un *screenshot* donde se evidencie la instalación (actualización) del paquete.

- Realice un commit+push para actualizar el repositorio.
- Actualice el repositorio en su máquina. Incluya en su reporte un pantallazo del archivo `model_abandono-0.0.2-py3-none-any.whl` en su máquina local.

Rúbrica

A continuación se presenta la rúbrica de calificación, con un resumen de cada punto considerado. La descripción completa de cada instrucción la puede encontrar en el numeral asociado.

Numeral	Evidencia	Puntaje
3.1	Pantallazo de la consola de AWS EC2 con la máquina en ejecución.	5
3.3	Pantallazo de conexión a la máquina virtual.	5
3.12	Pantallazo de la salida del comando <code>tox run -e train</code>	10
3.13	Pantallazo de la salida del comando <code>tox run -e test-package</code>	10
3.14	Pantallazo con archivos <code>tar.gz</code> y <code>whl</code> generados en la carpeta <code>dist</code> .	10
3.17	Pantallazo donde se evidencien los paquetes con el paquete del modelo instalado.	15
3.18	Pantallazo de ejecución del script <code>test-package.py</code> .	10
4.1	Pantallazo de la modificación realizada al archivo <code>config.yml</code>	10
4.4	Pantallazo del archivo <code>VERSION</code> actualizado en la máquina virtual.	10
4.5	Pantallazo donde se evidencie la instalación del paquete actualizado.	10
4.7	Pantallazo del archivo <code>model_abandono-0.0.2-py3-none-any.whl</code> en su máquina local.	5